



Home



My Network



Jobs



Messaging



Notifications



Me



For Business

Try Pre



In-Depth Understanding of AI Distributed Training Communication Primitives



NADDOD
2,575 followers



January 8, 2026

In large-scale artificial intelligence (AI) model training, single-machine, single-GPU setups are no longer sufficient to meet the demands for computing power and memory. As the scale of model parameters continues to expand, distributed training has become an inevitable path for the development of deep learning. The most fundamental capability supporting distributed training is distributed training communication primitives.

This article will explain what distributed training communication primitives are, outline some of the most common communication patterns, and help you understand their role and value in practical AI training.

What are Distributed Training Communication Primitives?

"Communication primitives" don't refer to a specific framework or API, but rather a set of highly abstract, recurring fundamental communication and coordination operations in various distributed systems. They define how multiple devices and processes exchange data, synchronize states, and maintain consistency during training. Whether you're using PyTorch, TensorFlow, or JAX, and regardless of whether you employ data parallelism, model parallelism, or hybrid parallelism, the

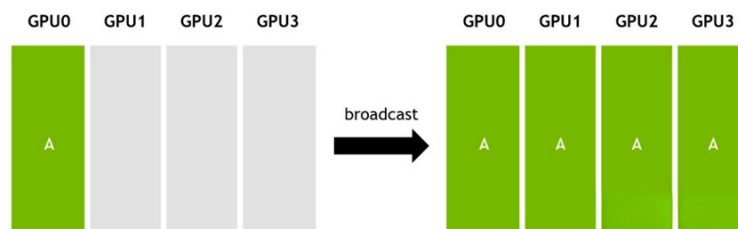
underlying implementation relies on the efficient implementation of these AI distributed training communication primitives. In short, the performance of communication primitives directly determines the upper limit of distributed training efficiency.

Detailed Explanation of Common Distributed Training Communication Primitives

Primitives Below, we introduce eight core AI training communication primitives based on the most common communication patterns encountered in actual training: Broadcast, Scatter, Gather, AllGather, Reduce, ReduceScatter, AllReduce, and All-To-All.

Broadcast

Broadcast is a one-to-many communication primitive, meaning one sender and multiple receivers. In a cluster, a node (or GPU) broadcasts its data to all other nodes. For example, synchronizing data from GPU0 to GPU1, GPU2, and GPU3.



Broadcast is a one-to-many data synchronization mechanism that synchronizes data from one GPU to all other GPUs. Its applications include:

- Parallel data parameter initialization, ensuring consistent initial parameters across all GPUs;
- Broadcast operations within the broadcast + reduce combination in allReduce;
- Broadcast operations in a distributed training parameter server architecture where the master node broadcasts data to

worker nodes, and then the worker nodes reduce data back to the master node.

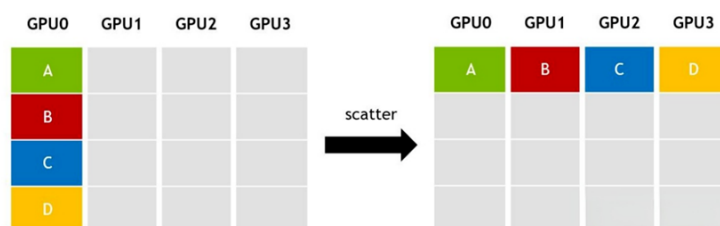
Scatter

Scatter and broadcast are both one-to-many communication primitives, but they are fundamentally different. The main difference between scatter and broadcast are:

- Broadcast: Sends the same complete data to all nodes.
- Scatter: Splits the data and sends the split data to different nodes.

For example, a piece of data DATA is split into DATA-A, DATA-B, DATA-C, and DATA-D:

- DATA-A → GPU0
- DATA-B → GPU1
- DATA-C → GPU2
- DATA-D → GPU3



The reverse operation of Scatter is Gather, and typical application scenarios include:

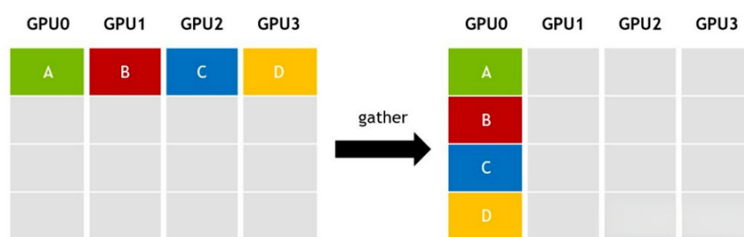
- The Scatter operation in ReduceScatter combinations;
- Scattering the model across different GPUs during initialization in model parallelism.

Gather

Gather is a many-to-one communication primitive with multiple data senders and one data receiver. It collects data from multiple nodes within a cluster onto a single node. Each node sends its data to the master node, and after the gather is complete, the master node possesses the data from all nodes. For example:

- GPU0 → DATA-A
- GPU1 → DATA-B
- GPU2 → DATA-C
- GPU3 → DATA-D

Ultimately, the master node holds the complete DATA-A/B/C/D.



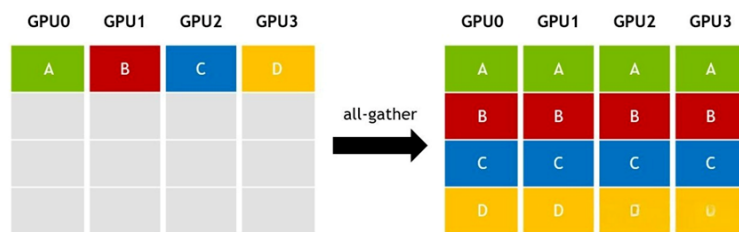
Gather is a many-to-one data collection method that collects data from multiple GPU cards onto a single GPU card. Its reverse operation corresponds to Scatter, and its application scenario is the Scatter operation in the ReduceScatter combination.

AllGather

AllGather is a many-to-many communication primitive that performs synchronous full collection of data across multiple GPU cards. It can be viewed as a combination of Gather and Broadcast operations, i.e.:

- First, collect data from all nodes onto the master node (Gather).
- Then, broadcast the complete data to all nodes (Broadcast).

The final result of Allgather is that every node has the complete data.



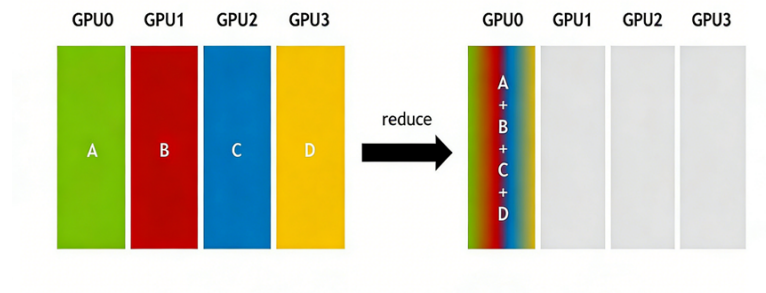
The reverse operation of AllGather is ReduceScatter, and typical application scenarios include:

- Parallel model training;
- In parallel model training, the parameters in the forward computation need to be fully synchronized. AllGather is needed to synchronize the parameters that were split across different GPUs onto a single GPU before forward computation can be performed.

Reduce

Reduce is a many-to-one communication primitive used to reduce data from multiple nodes and summarize the results to a single node. Common reduction operations include:

- Summation (SUM)
- Product (PROD)
- Maximum/Minimum (MAX/MIN)
- Logical AND/OR (LAND/LOR)
- Bitwise AND/OR/XOR (BAND/BOR/BXOR)
- MAXLOC/MINLOC (value and position)



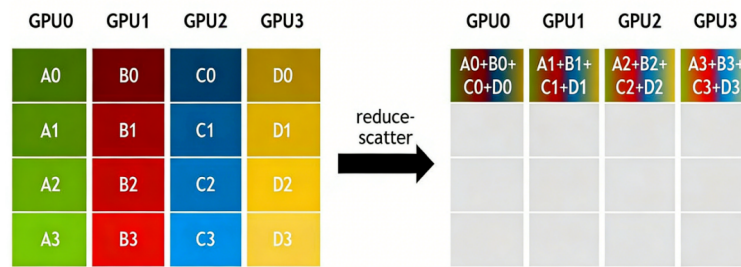
These reduction operations typically require GPU or accelerator card hardware support for the corresponding operators to achieve high performance.

Typical application scenarios for Reduce include:

- The Reduce phase in AllReduce;
- The Reduce operation in ReduceScatter combined communication;
- In a distributed training parameter server architecture, the Master node first broadcasts the model parameters to all Worker nodes; after each Worker completes its computation, the results are aggregated back to the Master node through a Reduce operation.

ReduceScatter

ReduceScatter is a many-to-many communication primitive with multiple data senders and multiple data receivers. On all nodes within the cluster, it performs a Reduce operation along the same dimensions, then distributes the result to every node in the cluster. ReduceScatter is equivalent to performing a Reduce operation once on each node, followed by a Scatter distribution operation. Its reverse operation is AllGather.

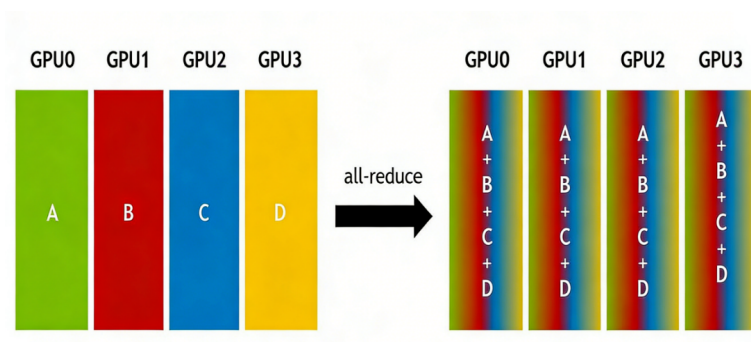


ReduceScatter is a many-to-many data operation that performs a "reduce-then-distribute" approach. It first reduces (e.g., sums) all data on the GPU cards, then distributes (scatters) the data. Its applications include:

- ReduceScatter can be applied to both data parallelism and model parallelism;
- The ReduceScatter operation within the ReduceScatter + Allgather combination in data parallelism;
- ReduceScatter in the reverse computation after the forward Allgather in model parallelism;

AllReduce

AllReduce is a many-to-many communication primitive, with multiple data senders and multiple data receivers. It performs the same Reduce operation on all nodes within the cluster and sends the reduction results from all nodes to each node. AllReduce can be implemented by executing Reduce + Broadcast or ReduceScatter + AllGather on the master node.

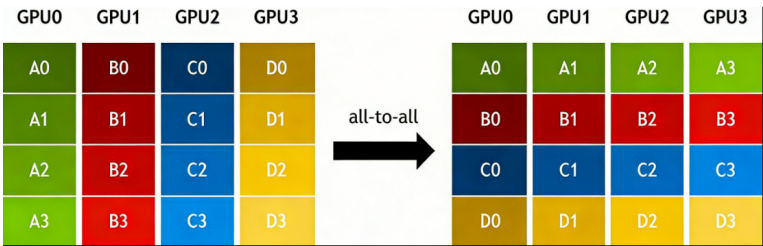


AllReduce is a many-to-many data reduction operation that reduces (e.g., sums) the data across all GPUs in the cluster to each GPU. Its application scenarios include:

- AllReduce can be applied to data parallelism;
- Data parallelism in various communication topologies, such as AllReduce in Ring AllReduce and Tree AllReduce.

All-to-All

In an All-To-All operation, data from each node is scattered to all nodes in the cluster, while each node also gathers data from all nodes in the cluster. All-To-All is an extension of All-Gather, the difference being that in All-Gather, different nodes collect the same data from a given node, while in All-To-All, different nodes collect different data from a given node, essentially performing a data transpose.



All-to-All is a many-to-many transpose operation that transposes data from all GPU cards to every GPU card in the cluster. Its main application scenarios include:

- All-to-All applied to model parallelism;
- Matrix transpose in model parallelism;
- Matrix transpose from data parallelism to model parallelism;

From Communication Primitives to Communication Libraries: Taking NCCL as an Example

NCCL (NVIDIA Collective Communications Library) is a library providing primitives for inter-GPU communication. It is topology-aware and easily integrated into various applications. NCCL implements Collective Communication and point-to-point send/receive primitives. It is not a complete parallel

programming framework, but rather a toolkit focused on accelerating inter-GPU communication.

NCCL provides Collective Communication primitives including AllReduce, Broadcast, Reduce, AllGather, ReduceScatter, AlltoAll, Gather, and Scatter. Furthermore, NCCL supports point-to-point send/receive communication, allowing for flexible implementation of Scatter, Gather, or All-to-All operations.

Through NCCL, communication primitives are conceptually translated into usable library functions. In traditional CUDA programming, Collective Communication is typically implemented through a combination of multiple CUDA memory copy operations and local reduction computation kernels; NCCL encapsulates each Collective Communication primitive in a single kernel, handling both communication and computation simultaneously, thus achieving more efficient synchronization and reducing resource overhead.

Conclusion

In distributed deep learning training, the training framework itself typically doesn't directly manipulate the underlying communication network. Instead, it uses high-performance communication libraries to handle operations such as parameter synchronization and gradient reduction. These libraries shield complex hardware details and improve training performance. Different AI chip and accelerator card manufacturers provide their own proprietary communication libraries or extensions, such as hardware-aware MPI implementations or self-developed CCL libraries, to optimize the use of the underlying network.

In real-world distributed training clusters, network interconnection patterns are extremely diverse. They may use common protocols like Ethernet, InfiniBand, and RoCE v1/v2, or proprietary or semi-proprietary high-speed interconnect protocols like NVLink. This necessitates communication libraries that can integrate with vendor-provided SDKs and interfaces, optimizing for specific hardware, such as CUDA-aware MPI, NCCL, and NVSHMEM. Furthermore, they must select the most suitable communication strategy and algorithm based on the actual network topology and node layout.

Comments



 Comment

Add a comment...



Be the first to comment.

Start the conversation

Follow to never miss an update.



Follow



English (English)

Learn more about Recommended Content.